Open Source Audit: A Deep Dive into Git

Student Name: Ashish Ranjan Pandey

Registration Number: 24BCE10812

Course: Open Source Software (OSS-NGMC)

Software Choice: Git (Distributed Version Control System)

1. Executive Summary

This audit explores Git, the world's most popular distributed version control system. While most developers use Git daily, few understand the volatile circumstances of its birth or the rigid ethical framework (GPL v2) that protects it. This report analyzes Git's origin story, its technical footprint on the Linux operating system, its role in the global FOSS ecosystem, and a critical comparison against proprietary alternatives. Furthermore, five automation scripts are provided to demonstrate practical Linux administration skills.

2. Part A: Origin and Philosophy

A1. The Problem: The 2005 BitKeeper Crisis

To understand why Git exists, we have to look at the "dark ages" of Linux kernel development. From 1991 to 2002, the Linux kernel was managed using simple patches and tarballs—a chaotic and slow process. In 2002, the kernel team started using a proprietary tool called BitKeeper. BitKeeper was efficient, but it was "free" only as long as you didn't try to build a competing tool. In 2005, a developer named Andrew Tridgell (the creator of Samba) began reverseengineering the BitKeeper protocol to create a free client. In retaliation, Larry McVoy (the CEO of BitKeeper) revoked the free license for all Linux developers. Linus Torvalds' Response: Suddenly, the world's most important open-source project had no way to manage its code. Linus Torvalds didn't just want a replacement; he wanted a tool that was the exact opposite of BitKeeper. He famously stated that existing tools like CVS or Subversion were "the wrong way to do things." He wanted: 1. Speed: It had to be fast enough to handle thousands of patches. 2. Distributed Nature: No central server should be a single point of failure. 3. Data Integrity: Protection against corruption (leading to the use of SHA-1 hashing). He wrote the first version of Git in a single weekend. The name "Git" itself is a British slang for a "contemptible person"—Linus jokingly said he names all his projects after himself (first "Linux," then "Git"). "In my opinion, the BitKeeper fallout was a massive wake-up call for the entire industry. It showed me that as a developer, your tools are your lifeblood; if you don't have the legal right to see the code or modify your own version control system, you're essentially building a house on rented land that can be reclaimed by a landlord at any moment."

A2. The License: Decoding the GPL v2

Git is licensed under the GNU General Public License, version 2 (GPL v2). This isn't just a legal document; it's a "Constitution" for the software.

The Four Freedoms

Under the GPL v2, Git grants users four essential freedoms: 1. To run the program for any purpose. 2. To study how the program works (access to source code). 3. To redistribute copies to help others. 4. To improve the program and release your improvements.

Why GPL v2 instead of MIT?

Linus Torvalds has often defended his choice of GPL v2 over more "permissive" licenses like MIT or BSD. The key difference is Copyleft. If a company takes Git's code and adds a amazing new feature, the GPL v2 forces them to share that code back with the community if they distribute it. If Git were MIT-licensed, a company like Microsoft or Google could have taken Git, made a "Pro" version, and kept the improvements secret. The GPL v2 ensures that the "Giants" must play by the same rules as the "Students."

A3. The Ethics of the "Free" Labor

Open source is often called "free as in speech, not free as in beer." While Git is free to download, it represents millions of dollars worth of volunteer labor. The Standing on Shoulders of Giants Argument: Today, companies like GitHub (owned by Microsoft) and GitLab have built multi-billion dollar businesses on top of Git. Is it ethical for a corporation to profit so heavily from a tool they didn't create? The counter-argument is that these companies contribute back by providing hosting and ecosystem tools that make Git accessible to non-technical people. However, as an auditor, one must reflect on the power imbalance. If GitHub goes down, much of the world's opensource development stops, even though Git itself is decentralized. This "centralization of the decentralized" is a modern ethical dilemma in the FOSS community.

3. Part B: Linux Footprint

B1. Installation and Management

On a standard Linux system (like the Ubuntu environment used for this audit), Git is typically installed via the apt package manager:

sudo apt update

sudo apt install git

Key Directories (Where Git "Lives")

1. /usr/bin/git: The primary binary file.

2. /etc/gitconfig: System-wide configuration settings.

3. ~/.gitconfig: User-specific settings (Name, Email, Aliases).

4. /usr/share/doc/git: Documentation and man pages.

User and Group Management

Unlike a database (MySQL) or a web server (Apache), Git does not usually run as a background service or “daemon.” It doesn’t have its own “git” user on a local machine; it runs with the permissions of whichever user executes the command. This makes it incredibly secure from a service-vulnerability perspective, but it means local file permissions (read/write) are the only thing protecting a repository

4. Part C: The FOSS Ecosystem

C1. Dependencies and Shared Libraries

Git does not stand alone. It is a master orchestrator of many other open-source libraries. For example: - zlib: Used for compressing your objects (blobs, trees, and commits). - curl: Used for interacting with HTTP and HTTPS protocols (like when you git clone from GitHub). - OpenSSL: Provides the secure encryption layer for data in transit. - Expat: A XML parser library used for some of Git’s legacy protocols.

C2. Its Role in the LAMP Stack

While Git isn’t strictly part of the “LAMP” (Linux, Apache, MySQL, PHP/Python) acronym, it has become the “Glue” that holds it together. Today, most Apache web servers are deployed directly from a Git repository using “hooks.”

C3. Community Governance

Git is governed through the Git Mailing List. Unlike proprietary software where a product manager decides the roadmap, Git's direction is determined by a core team of maintainers (currently led by Junio Hamano) who review patches submitted by anyone from around the world. This is "meritocracy" in its purest form.

5. Part D: Critical Comparison (Open vs. Proprietary)

Comparison Table: Git (Open) vs. Perforce Helix Core (Proprietary)

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	$0. Community supported.	Expensive. High licensing fees per user.
Security	Transparent.	Opaque.
Reliability High.	Decentralized;	Centralized.
Freedom to Modify	Total.	Locked.
Learning Curve	Steep.	Moderate.

Overall Verdict:

Perforce is often preferred by large gaming studios because it handles massive binary files (like 3D models) better than Git. However, for source code and general software development, Git's decentralized nature and lack of licensing fees make it the clear winner for almost any open-source or modern agile team.

6. Part E: Shell Script Implementation

As part of this audit, five shell scripts were developed to demonstrate practical mastery of the Linux command line. These scripts automate system reporting, package inspection, and security auditing.

Script 1: System Identity Report

This script introduces the Linux environment and provides a snapshot of the current system state.

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Ashish Ranjan Pandey

# — ANSI Colors ————————————————————————————————————

BOLD="\033[1m"
```

```
 CYAN="\033[1;36m"

GREEN="\033[1;32m"

YELLOW="\033[1;33m"

 RESET="\033[0m"

STUDENT_NAME="Ashish Ranjan Pandey"

SOFTWARE_CHOICE="Git"

# ── Gather System Info ──────────────────────────────────────────

KERNEL=$(uname -r)

ARCH=$(uname -m)

USER_NAME=$(whoami)

UPTIME=$(uptime -p 2>/dev/null || uptime)

DATE=$(date '+%A, %d %B %Y — %H:%M:%S')

DISTRO=$(grep PRETTY_NAME /etc/os-release 2>/dev/null | cut -d '"' -f2 || echo "Unknown")
CPU=$(grep -m1 "model name" /proc/cpuinfo 2>/dev/null | cut -d ':' -f2 | xargs || echo "N/A")
RAM_KB=$(grep MemTotal /proc/meminfo 2>/dev/null | awk '{print $2}')

RAM_GB=$(awk "BEGIN {printf \"%.1f GB\", $RAM_KB/1048576}" 2>/dev/null || echo "N/A")
GIT_VER=$(git --version 2>/dev/null || echo "Git not installed")

GIT_USER=$(git config --global user.name 2>/dev/null || echo "Not configured")

 # ── Output ──────────────────────────────────────────────────────

echo -e "${CYAN}${BOLD}"

 echo "╔═══════════════════════════════════════════════════════════════╗"

 echo "│ Open Source Audit — System Report                             │"

 echo "╚═══════════════════════════════════════════════════════════════╝"

echo -e "${RESET}"

printf "${BOLD}%-16s${RESET} %s\n" "Student" "$STUDENT_NAME"

 printf "${BOLD}%-16s${RESET} %s\n" "Software" "$SOFTWARE_CHOICE"

echo -e

"${CYAN}──────────────────────────────────────────────────${RESET}"

printf "${BOLD}%-16s${RESET} %s\n" "Date" "$DATE"
```

```
 printf "${BOLD}%-16s${RESET} %s\n" "User" "$USER_NAME"
printf "${BOLD}%-16s${RESET} %s\n" "Home Dir" "$HOME_DIR"
 printf "${BOLD}%-16s${RESET} %s\n" "Distro" "$DISTRO"
printf "${BOLD}%-16s${RESET} %s\n" "Kernel" "$KERNEL"
printf "${BOLD}%-16s${RESET} %s\n" "Architecture" "$ARCH"
printf "${BOLD}%-16s${RESET} %s\n" "CPU" "$CPU"
 printf "${BOLD}%-16s${RESET} %s\n" "RAM" "$RAM_GB"
printf "${BOLD}%-16s${RESET} %s\n" "Uptime" "$UPTIME"
echo -e
"${CYAN}──────────────────────────────────────────────────────${RESET}"
printf "${BOLD}%-16s${RESET} %s\n" "Git Version" "$GIT_VER"
 printf "${BOLD}%-16s${RESET} %s\n" "Git User" "$GIT_USER"
printf "${BOLD}%-16s${RESET} %s\n" "Kernel Lic." "GPL v2"
echo ""
echo -e "${GREEN}✔ System identity report complete.${RESET}"
```

Script 2: Package Inspector

A cross-distribution tool that checks for the presence of software (defaulting to Git).

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author : Ashish Ranjan Pandey
# ── ANSI Colors ──────────────────────────────────────────────
BOLD="\033[1m"
GREEN="\033[1;32m"
RED="\033[1;31m"
CYAN="\033[1;36m"
YELLOW="\033[1;33m"
RESET="\033[0m"
```

```
# — Package Selection (CLI arg or default: git) ————————————
PACKAGE="${1:-git}"
# — Detect Package Manager ————————————————————————————
detect_pkg_manager() {
 if command -v apt &>/dev/null; then echo "apt"
 elif command -v dnf &>/dev/null; then echo "dnf"
 elif command -v rpm &>/dev/null; then echo "rpm"
else echo "unknown"
 fi
}
PKG_MGR=$(detect_pkg_manager)
 # — Check if package is installed ————————————————————
is_installed() {
case "$PKG_MGR" in
apt) dpkg -l | grep -qw "$1" ;;
dnf|rpm) rpm -q "$1" &>/dev/null ;;
*) command -v "$1" &>/dev/null ;;
 Esac
 }
 # — Get package info ————————————————————————————————
get_pkg_info() {
 case "$PKG_MGR" in
apt) apt show "$1" 2>/dev/null | grep -E
'^(Version|Maintainer|Homepage|Description)' ;;
dnf) dnf info "$1" 2>/dev/null | grep -E '(Version|Summary|URL)' ;;
rpm) rpm -qi "$1" 2>/dev/null | grep -E '(Version|Summary|URL)' ;;
*) echo " Package manager not supported for info lookup." ;;
```

```
 Esac
 }

# — FOSS package descriptions ————————————————————————————
foss_note() {
 case "$1" in git) echo "Git: decentralized version control, the backbone of open-source collaboration." ;;
apache2|httpd) echo "Apache: battle-tested open web server powering ~30% of the internet." ;;
 mysql) echo "MySQL: open source relational database used in millions of applications." ;;
firefox) echo "Firefox: privacy-focused open web browser by Mozilla Foundation." ;;
 vim) echo "Vim: powerful, extensible terminal text editor (GPL licensed)." ;;
 python3) echo "Python: interpreted, high-level, open-source general-purpose language." ;;
*) echo "${1^}: an open-source software package." ;;
Esac
 }

# — Header ————————————————————————————————————————————
echo -e "${CYAN}${BOLD}"
echo "  ╔═══════════════════════════════════════════════════════════════════════════╗"
echo "  ║  FOSS Package Inspector                                                   ║"
 echo "  ╚═══════════════════════════════════════════════════════════════════════════╝"
echo -e "${RESET}"
 echo -e "${BOLD}Package Manager:${RESET} $PKG_MGR"
echo -e "${BOLD}Target Package :${RESET} $PACKAGE"
 echo –e
 "${CYAN}————————————————————————————————————————————${RESET}"
 # — Installation Status ————————————————————————————————
if is_installed "$PACKAGE"; then
```

```
echo -e "${GREEN}${BOLD}✔ $PACKAGE is INSTALLED${RESET}"

echo ""

echo -e "${BOLD}Package Details:${RESET}" get_pkg_info "$PACKAGE" else

echo -e "${RED}${BOLD}✘ $PACKAGE is NOT installed${RESET}"

echo -e "${YELLOW} Install with: ${PKG_MGR} install ${PACKAGE}${RESET}"

fi

echo -e

"${CYAN}────────────────────────────────────────────────────${RESET}"

 echo -e "${BOLD}About this FOSS package:${RESET}"

echo " $(foss_note "$PACKAGE")"

# — Live Git Stats (only if package is git and git exists) ——

if [[ "$PACKAGE" == "git" ]] && command -v git &>/dev/null; then

echo ""

 echo -e "${CYAN}────────────────────────────────────────────────────${RESET}"

echo -e "${BOLD}Git Runtime Info:${RESET}”

printf " ${BOLD}%-20s${RESET} %s\n" "Version" "$(git --version)"

printf " ${BOLD}%-20s${RESET} %s\n" "Config user" "$(git config --global user.name 2>/dev/null ||
echo 'Not set')"

printf " ${BOLD}%-20s${RESET} %s\n" "Config email" "$(git config --global user.email 2>/dev/null ||
echo 'Not set')"

printf " ${BOLD}%-20s${RESET} %s\n" "Default branch" "$(git config --global init.defaultBranch
2>/dev/null || echo 'master')"

# Show recent commits if inside a git repo

 if git rev-parse --is-inside-work-tree &>/dev/null; then

echo ""

echo -e "${BOLD} Last 5 commits in current repo:${RESET}"

git log --oneline -5 2>/dev/null | sed 's/^/ /'

 fi

 fi
```

```
echo ""

 echo -e "${GREEN}✔ Package inspection complete.${RESET}"
```

Script 3: Disk & Permission Auditor

A security-focused script that scans critical system directories and evaluates permissions.

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

# Author: Ashish Ranjan Pandey

 # ── ANSI Colors ──────────────────────────────────────────────

 BOLD="\033[1m"

GREEN="\033[1;32m"

RED="\033[1;31m" CYAN="\033[1;36m"

YELLOW="\033[1;33m" RESET="\033[0m" DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

 # ── Header ───────────────────────────────────────────────────

echo -e "${CYAN}${BOLD}"

echo "╔═══════════════════════════════════════════════════════════════════╗"

echo "║ Disk & Permission Auditor                                         ║"

echo "╚═══════════════════════════════════════════════════════════════════╝"

 echo -e "${RESET}"

 # ── Directory Audit Table ────────────────────────────────────

printf "${BOLD}%-15s %-25s %-8s %-10s %-10s${RESET}\n" \ "Directory" "Permissions / Owner"
"Group" "Size" "Status"

 echo –e

 "${CYAN}──────────────────────────────────────────────────────
──────────────${RESET}"

for DIR in "${DIRS[@]}"; do

 if [ -d "$DIR" ]; then

PERMS=$(ls -ld "$DIR" | awk '{print $1}')

OWNER=$(ls -ld "$DIR" | awk '{print $3}')
```

```
GROUP=$(ls -ld "$DIR" | awk '{print $4}')

SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

printf "%-15s ${GREEN}%-14s${RESET} %-10s %-10s %-10s ${GREEN}OK${RESET}\n" \ "$DIR"
"$PERMS $OWNER" "$GROUP" "$SIZE" else

 printf "%-15s ${RED}%-34s${RESET} ${RED}MISSING${RESET}\n" "$DIR" "Directory does not exist" fi
done

 echo -e

"${CYAN}──────────────────────────────────────────────────────────── ────────────────${RESET}"
# ── World-Writable Directory Warning ──────────────────────────────

echo ""

echo -e "${BOLD}World-Writable Directory Check:${RESET}"

WW_DIRS=$(find /tmp /var/tmp -maxdepth 1 -perm -0002 -type d 2>/dev/null)

if [ -n "$WW_DIRS" ]; then

echo -e "${YELLOW} ⚠ World-writable directories found:${RESET}"

 echo "$WW_DIRS" | sed 's/^/ /' else echo -e "${GREEN}

✔ No unexpected world-writable directories found.${RESET}"

fi

# ── Git Conβig Check ──────────────────────────────────────────────

echo ""

echo -e "${BOLD}Git Configuration Check:${RESET}"

CONFIG_DIR="$HOME/.gitconfig"

if [ -f "$CONFIG_DIR" ]; then

 echo -e "${GREEN} ✔ Git config found: $CONFIG_DIR${RESET}"

printf " ${BOLD}%-20s${RESET} %s\n"

"Permissions:" "$(ls -l "$CONFIG_DIR" | awk '{print $1, $3, $4}')"

 printf " ${BOLD}%-20s${RESET} %s\n"

 "Last modified:" "$(stat -c '%y' "$CONFIG_DIR" 2>/dev/null | cut -d'.' -f1)"

 else
```

```
echo -e "${YELLOW} ⚠ Git config not found at $CONFIG_DIR${RESET}"

 fi

 # — Git Repository Scan ————————————————————————————

echo ""

echo -e "${BOLD}Git Repository Scan (home directory):${RESET}"

GIT_REPOS=$(find "$HOME" -maxdepth 4 -name ".git" -type d 2>/dev/null | sed 's|/.git$||')
REPO_COUNT=$(echo "$GIT_REPOS" | grep -c '.' 2>/dev/null || echo 0)

if [ "$REPO_COUNT" -gt 0 ]; then

echo -e "${GREEN} ✔ Found $REPO_COUNT git repo(s):${RESET}"

 echo "$GIT_REPOS" | head -10 | sed 's/^/ /'

 [ "$REPO_COUNT" -gt 10 ] && echo " ... and $((REPO_COUNT - 10)) more."

else

echo -e "${YELLOW} No git repositories found in $HOME${RESET}"

fi

 echo ""

echo -e "${GREEN}✔ Disk audit complete.${RESET}"
```

Script 4: Log Analyzer

A performance-optimized script (using while-read loops and grep) to scan system logs for errors.

```
#!/bin/bash

# Script 4: Log File Analyzer

# Author: Ashish Ranjan Pandey

# — ANSI Colors ————————————————————————————————————

 BOLD="\033[1m"

GREEN="\033[1;32m"

RED="\033[1;31m" CYAN="\033[1;36m"

YELLOW="\033[1;33m"

RESET="\033[0m"

 # — Usage ————————————————————————————————————————
```

```
usage() {
 echo -e "${BOLD}Usage:${RESET} $0 [keyword]"
echo ""
 echo " Path to the log file to analyze"
echo " [keyword] Optional keyword to search (default: error)"
 echo ""
 echo -e "${BOLD}Examples:${RESET}"
 echo " $0 /var/log/syslog"
echo " $0 /var/log/syslog warning" exit 1
}

# — Argument Validation ————————————————————————————————
if [ -z "$1" ]; then
usage
fi
 LOGFILE="$1"
KEYWORD="${2:-error}"
 if [ ! -f "$LOGFILE" ]; then
echo -e "${RED}${BOLD}✘ Error:${RESET} File not found — '$LOGFILE'"
echo " Please provide a valid log file path." exit 1
fi
 if [ ! -r "$LOGFILE" ]; then
 echo -e "${RED}${BOLD}✘ Error:${RESET} Permission denied reading '$LOGFILE'"
echo " Try: sudo $0 $LOGFILE $KEYWORD
exit 1
fi
 # — Header ————————————————————————————————————————————
```

```
echo -e "${CYAN}${BOLD}"
 echo "╔═══════════════════════════════════════════════════════════════════╗"
echo "│ Log File Analyzer                                                 │"
 echo "╚═══════════════════════════════════════════════════════════════════╝"
echo -e "${RESET}"
 printf "${BOLD}%-16s${RESET} %s\n" "Log File" "$LOGFILE"
printf "${BOLD}%-16s${RESET} %s\n" "File Size" "$(du -sh "$LOGFILE" | cut -f1)"
printf "${BOLD}%-16s${RESET} %s\n" "Total Lines" "$(wc -l < "$LOGFILE")"
printf "${BOLD}%-16s${RESET} %s\n" "Search Term" "$KEYWORD"
echo –e
 "${CYAN}──────────────────────────────────────────────────${RESET}"
# — Count using while-read loop (as required by assignment rubric) –
 COUNT=0
while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"
if [ "$COUNT" -gt 0 ]; then
echo -e "${BOLD}Primary Keyword:${RESET} ${YELLOW}'$KEYWORD'${RESET} found
${BOLD}${YELLOW}$COUNT${RESET} time(s)"
else
 echo -e "${BOLD}Primary Keyword:${RESET} ${GREEN}'$KEYWORD'${RESET} — ${GREEN}not
found${RESET} (log looks clean)"
 fi
 # — Severity Breakdown ──────────────────────────────────────
echo ""
echo -e "${BOLD}Severity Breakdown:${RESET}"
```

```
for LEVEL in error warning critical info debug; do
LEVEL_COUNT=$(grep -ci "$LEVEL" "$LOGFILE")
if [ "$LEVEL_COUNT" -gt 0 ]; then
case "$LEVEL" in error|critical) COLOR="${RED}" ;;
warning) COLOR="${YELLOW}" ;;
 *) COLOR="${GREEN}" ;;
esac
printf " ${COLOR}${BOLD}%-10s${RESET} %s occurrences\n" \ "[$LEVEL]" "$LEVEL_COUNT"
fi
done
# — Timestamps of First & Last Match ————————————————
echo ""
echo –e
 "${CYAN}————————————————————————————————————————${RESET}"
echo -e "${BOLD}First 5 matches for '${KEYWORD}':${RESET}"
 grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | head -5 | \
 awk '{print " "NR". "$0}' | cut -c1-100
echo ""
echo -e "${BOLD}Last 5 matches for '${KEYWORD}':${RESET}"
grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5 | \
 awk '{print " "NR". "$0}' | cut -c1-100
echo ""
echo -e "${GREEN}✔ Log analysis complete.${RESET}"
```

Script 5: Open Source Manifesto Generator

An interactive script that allows the user to declare their personal philosophy on free software.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
```

```
# Author: Ashish Ranjan Pandey

# — ANSI Colors ————————————————————————————————————————

BOLD="\033[1m"

GREEN="\033[1;32m"

CYAN="\033[1;36m"

YELLOW="\033[1;33m"

RESET="\033[0m"

# Note on Aliases: To run this script easily from anywhere, you could add

# an alias to your ~/.bashrc file, for example: alias manifesto='./script5_manifesto.sh'

# — Header ————————————————————————————————————————————

echo -e "${CYAN}${BOLD}"

echo "╔═══════════════════════════════════════════════════════════════════════════╗"

echo "│ Open Source Manifesto Generator                                           │"

echo "└═══════════════════════════════════════════════════════════════════════════╝"

echo -e "${RESET}" echo "Answer three questions to generate your personal open-source manifesto."

echo ""

# — Input with Validation (re-prompt on empty) ————————————

prompt_required() {

local PROMPT="$1"

local VAR=""

while [ -z "$VAR" ]; do

read -p "$(echo -e "${BOLD}${PROMPT}${RESET} ")" VAR

if [ -z "$VAR" ]; then

echo -e "${YELLOW} ⚠ This field cannot be empty. Please answer.${RESET}"

fi done

echo "$VAR"

}
```

```
TOOL=$(prompt_required "Open-source tool you use:")

FREEDOM=$(prompt_required " What does freedom mean to you:")

BUILD=$(prompt_required " What will you build & share:")

# ── Output File ──────────────────────────────────────────────

DATE=$(date '+%d %B %Y')

OUTPUT="manifesto_$(whoami).txt"

# ── Overwrite Guard ──────────────────────────────────────────

if [ -f "$OUTPUT" ]; then

echo "" echo -e "${YELLOW}⚠ A manifesto already exists: ${BOLD}$OUTPUT${RESET}"

read -p "$(echo -e "${BOLD}Overwrite it? [y/N]: ${RESET}")" CONFIRM

CONFIRM="${CONFIRM,,}" # lowercase

 if [[ "$CONFIRM" != "y" && "$CONFIRM" != "yes" ]]; then




echo -e "${CYAN} Manifesto not overwritten. Exiting.${RESET}"

exit 0

fi

 fi

 # ── Write Manifesto ──────────────────────────────────────────

{

echo "═══════════════════════════════════════════════════════════"

echo " MY OPEN SOURCE MANIFESTO"

echo "═══════════════════════════════════════════════════════════"

echo ""

echo "Author : $(whoami)"
```

```
echo "Date : $DATE"

echo ""

echo "On $DATE, I believe that open source is about $FREEDOM."

echo ""

echo "Tools like $TOOL empower developers worldwide, enabling"

echo "collaboration, transparency, and shared progress."

echo ""

echo "I aim to build $BUILD and share it freely with the world."

echo ""

echo "Because open source is not just code —"

echo "it is a philosophy of trust, community, and freedom."

echo ""

echo "═══════════════════════════════════════════════════════════════"

echo " Signed with conviction, $(whoami)"

echo "═══════════════════════════════════════════════════════════════"

 } > "$OUTPUT"

# — Display Result ──────────────────────────────────────────

echo ""

echo -e "${GREEN}${BOLD}✔ Manifesto saved to: ${OUTPUT}${RESET}"

echo ""

echo -e "${CYAN}──────────────────────────────────────────────${RESET}"

cat "$OUTPUT"

echo -e "${CYAN}──────────────────────────────────────────────${RESET}"

echo ""

echo -e "${GREEN}✔ Manifesto generation complete.${RESET}"
```

7. Conclusion

After a thorough audit of Git, its history, its license, and its technical footprint, it is clear that Git is not just a tool—it is the bedrock of modern software development. It embodies the core values of the FOSS community: transparency, resilience, and collaborative progress. For any real-world deployment, Git is the highly recommended choice for version control. I would absolutely contribute to its community because its “copyleft” license ensures that my contributions will always remain free and accessible to all future developers.